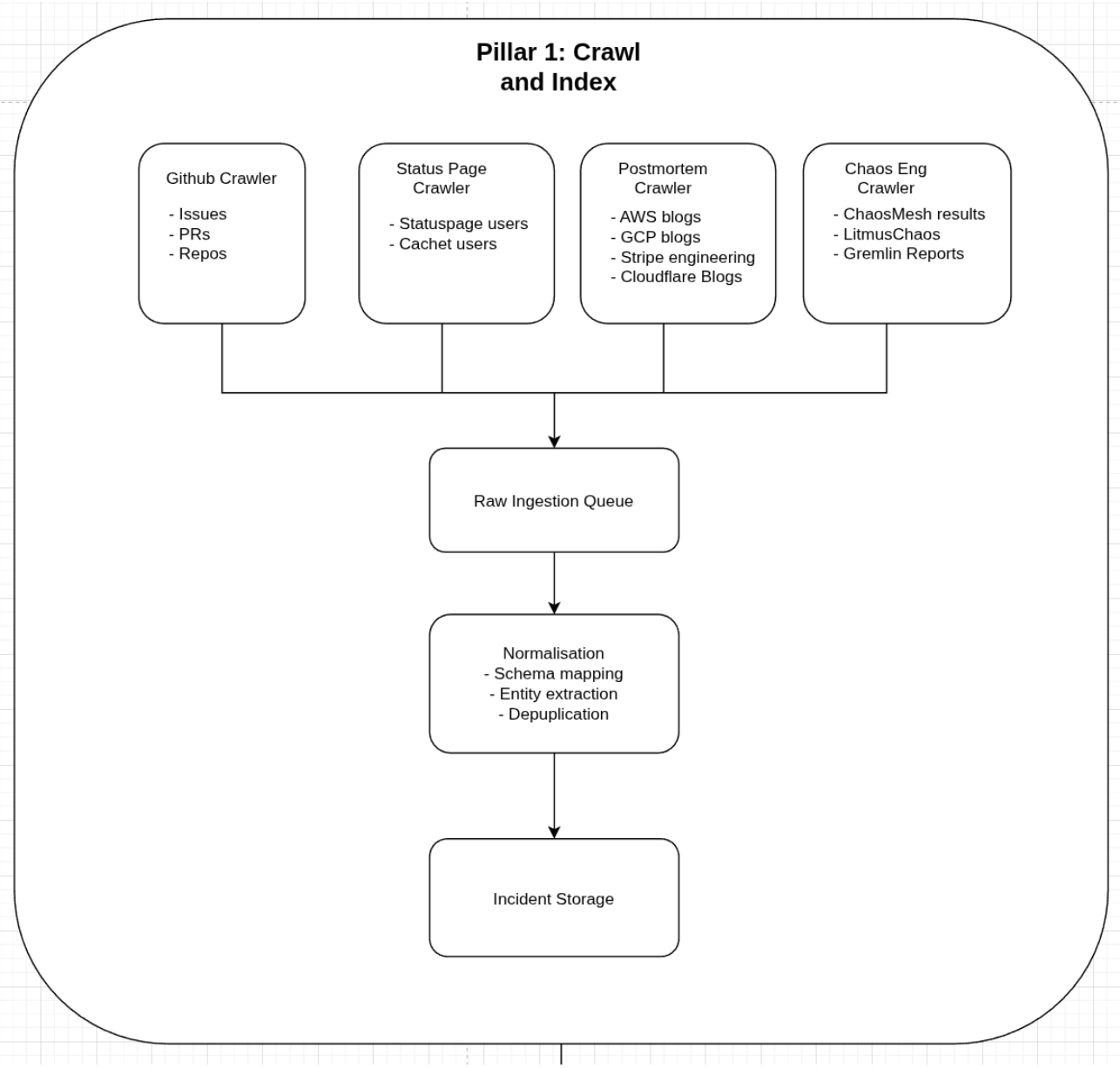
Pillar 1: Crawling & Indexing



Data Sources

Source Type	Concrete Examples	Update Frequency
GitHub post-mortems	github.com/danluu/post-mortems, org repos with /postmortems paths	Daily poll via GitHub API
Status page history	Atlassian Statuspage API, Cachet, self-hosted pages	Every 15 minutes
Engineering blogs	Cloudflare, AWS, GCP, Netflix, Stripe RSS/Atom feeds	Weekly
Chaos engineering	ChaosMesh reports, LitmusChaos results, public GameDay writeups	Weekly crawl
Public CVEs/advisories	NVD, OSS-Sec mailing list	Daily

Source Type	Concrete Examples	Update Frequency
Community reports	HackerNews "Ask HN: what broke" threads, r/devops	Daily

Ingestion Pipeline

Source → Crawler Worker → Raw Store (S3) → Parser → Normaliser → Quality Filter → Index

Crawlers are scheduled via a priority queue where sources with higher historical yield get more frequent polling.

The normaliser produces an `IncidentRecord`:

```
@dataclass class IncidentRecord:
>   id: str # content hash
>   sourceUrl: str
>   sourceType: SourceType # POSTMORTEM | STATUS_PAGE | BLOG | CHAOS_RESULT
>   title: str
>   description: str
>   timeline: list[TimelineEvent] # [{t: timestamp, event: str}]
>   affectedServices: list[str]
>   rootCauses: list[str] # raw text
>   remediationActions: list[str]
>   durationMinutes: int | None
    severity: Severity | None # P1..P5 normalized from source terminology
>   tags: list[str]
>   ingestedAt: datetime
>   contentHash: str # for dedup
>   qualityScore: float # 0..1
```

Deduplication

MinHash with Locality-Sensitive Hashing (LSH) on the description + root cause fields catches near-duplicate incidents reported across multiple sources (e.g., the same AWS outage reported on the AWS status page, three engineering blogs, and a GitHub post-mortem). Exact-duplicate detection uses `contenHash` on normalised text after stopword removal. We keep all variants linked by a canonical ID and use the highest-quality copy as the primary record; the others are retained as source authenticity.

Quality Filtering

A quality score (0–1) is computed from:

- **Completeness**: does it have a timeline? root cause identified? remediation described? (weighted 40%)
- **Specificity**: does it name specific components, error codes, metrics? (LLM-scored, 30%)

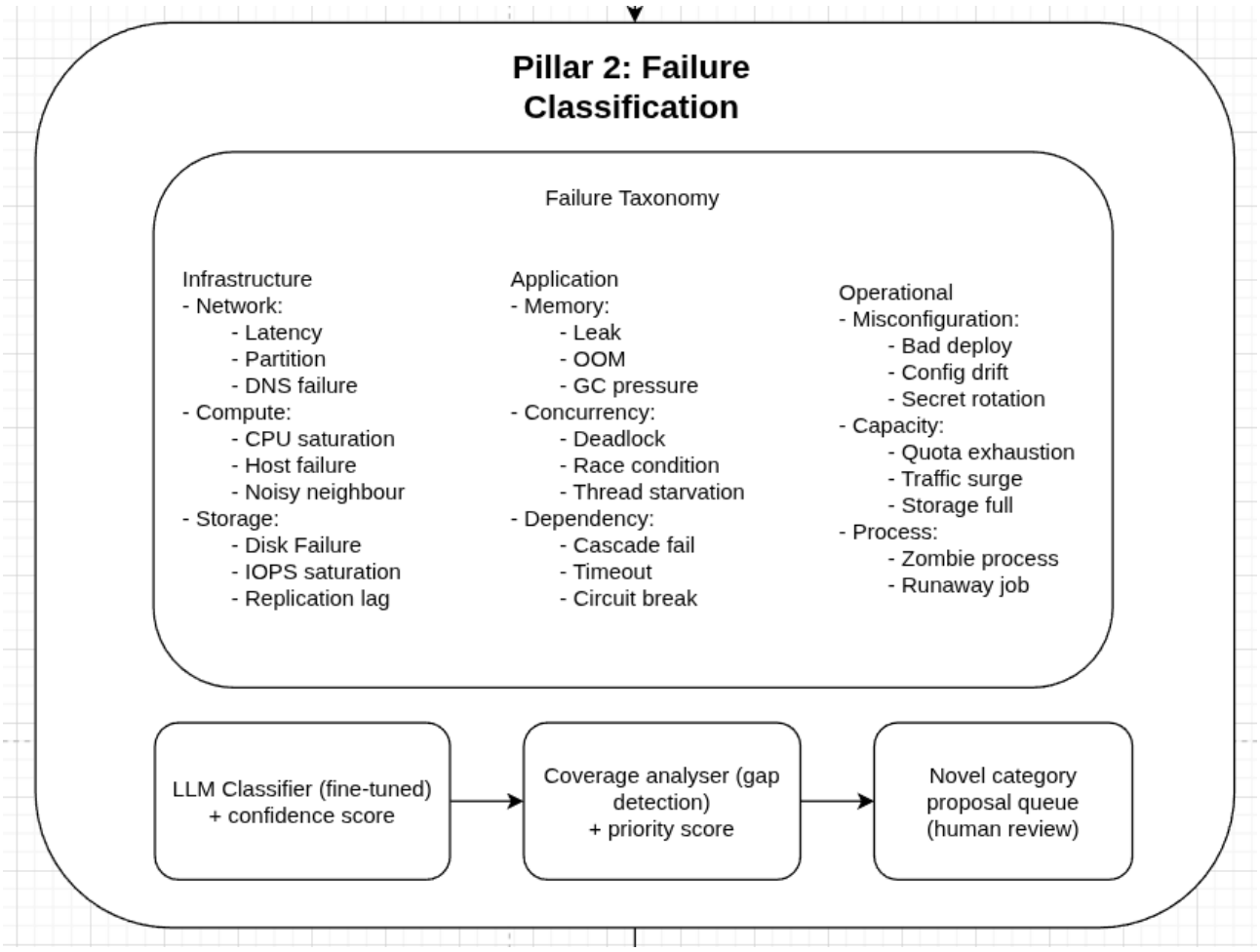
- **Actionability**: would this teach a diagnostic skill? (heuristic: mentions of monitoring signals, runbooks, 30%)

Records below 0.3 are archived but not used for scenario generation. Records 0.3–0.6 are used for classification training only. Records above 0.6 are used for scenario generation.

Staleness Handling

Each source has a configured TTL for re-crawling. Status page incidents are re-crawled until they show a "resolved" state. Blog posts are treated as immutable after ingestion.

Pillar 2 – Failure Classification and Taxonomy



Classification

Stage 1 - LLM classification: Model receives the incident description + root cause text and returns a ranked list of taxonomy nodes with confidence scores. Prompt includes the full taxonomy with brief descriptions of each leaf node.

Stage 2 - Rule-based verification: A deterministic rule engine checks for explicit signal patterns (error codes, metric names, infrastructure terms) that are hard evidence for specific taxonomy nodes. Rules override or boost LLM classifications.

Coverage Gap Detection

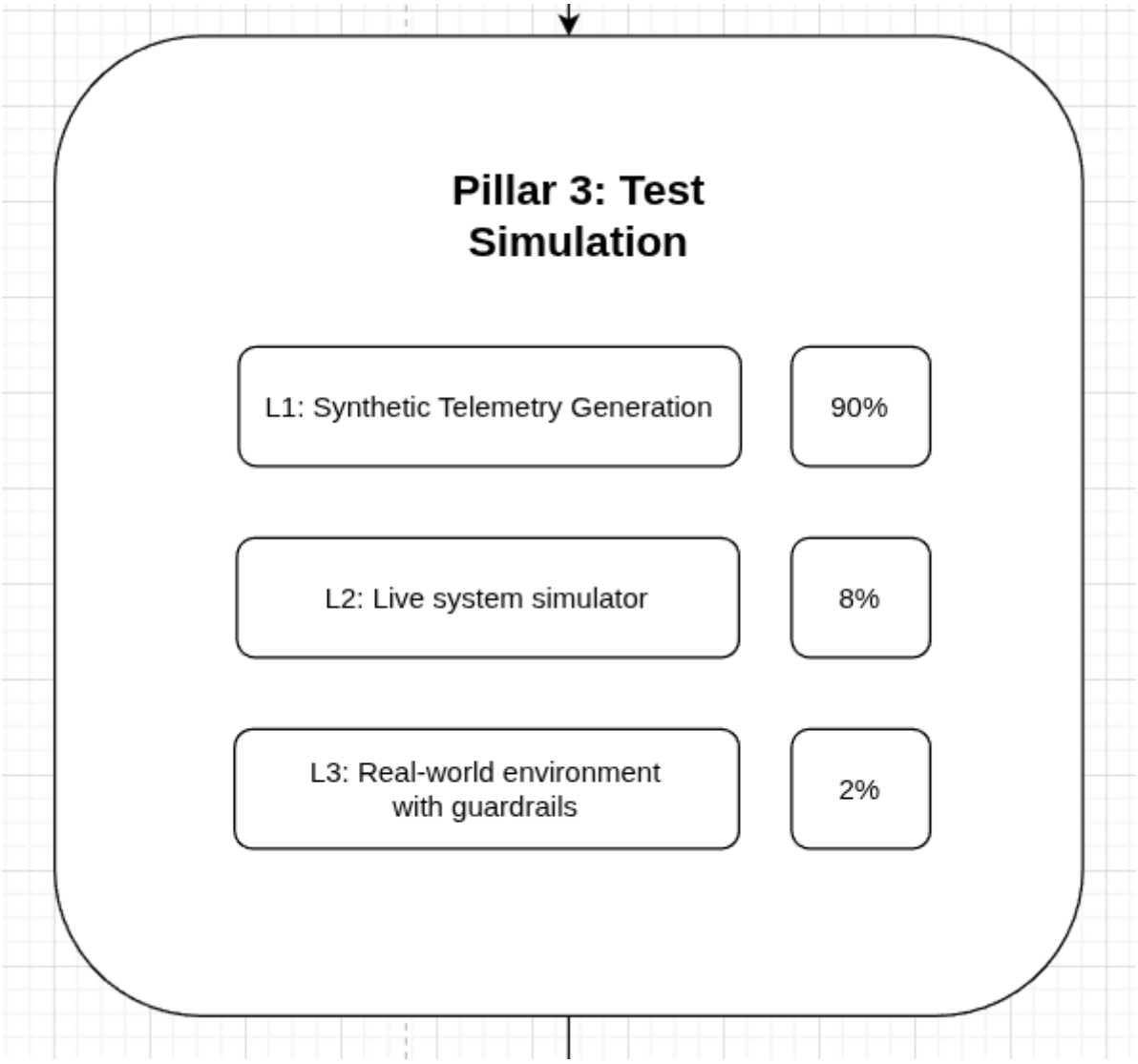
A coverage entropy score is computed for each taxonomy leaf node:

Nodes with low incident count, few generated scenarios, or low agent success rate receive a high gap priority score. A weekly report surfaces the top 10 under-represented nodes to drive targeted crawling and scenario synthesis.

Novel Incidents

Incidents that score below 0.4 confidence against all existing taxonomy nodes are routed to a Novel Incident Queue. A human reviewer (in the form of an expert SRE engineer) either maps it to an existing node (with notes explaining the mapping) or proposes a new leaf node. New nodes require at least 3 distinct incidents before being promoted to the taxonomy. This prevents taxonomy bloat from one-off events.

Pillar 3: Simulation of Distributed Infrastructure



I believe the best position to take is a hybrid approach to simulation training.

- Synthetic telemetry (L1) for the majority of training (80-90%)
- Container sandboxes (L2) for integration-class scenarios (5-15%)
- Real cluster simulation (L3) for the highest-fidelity validation scenarios (1-5%)

A reinforcement learning training loop needs thousands of episodes per hour; real infrastructure cannot provide that at reasonable cost, and waiting 5 minutes per episode makes the feedback loop too slow to evaluate.

Layer 1: Synthetic Telemetry Generation (80-90% of training episodes)

A telemetry generator that produces realistic logs, metrics, and traces without running any real infrastructure.

Strengths: ~100ms per episode generation, fully deterministic, easy to scale, zero infrastructure cost, easy to vary parameters for curriculum learning.

Weaknesses: Can't capture emergent behaviours from real system interactions. Telemetry patterns are modelled, not observed. Agent may learn to recognise synthetic artifacts rather than real failure signals.

Mitigation: Regularly validate L1 scenarios against L3 ground truth. Use real telemetry samples as training data for the generator. Include realistic noise, missing data, and instrumentation gaps.

Layer 2: Container Sandbox (5-15% of training episodes)

What it is: Lightweight containerised service meshes using Docker Compose or Podman, with fault injection via a sidecar agent. This layer is used for scenarios where the interaction between components matters and can't be adequately modelled synthetically.

Episode time: ~30 seconds spin-up + ~2 minutes scenario execution. Parallelised across a pool of sandbox workers.

Cost: Each worker is a ~4-core VM; a pool of 20 workers can run ~500 episodes/hour at ~$2/hour total.

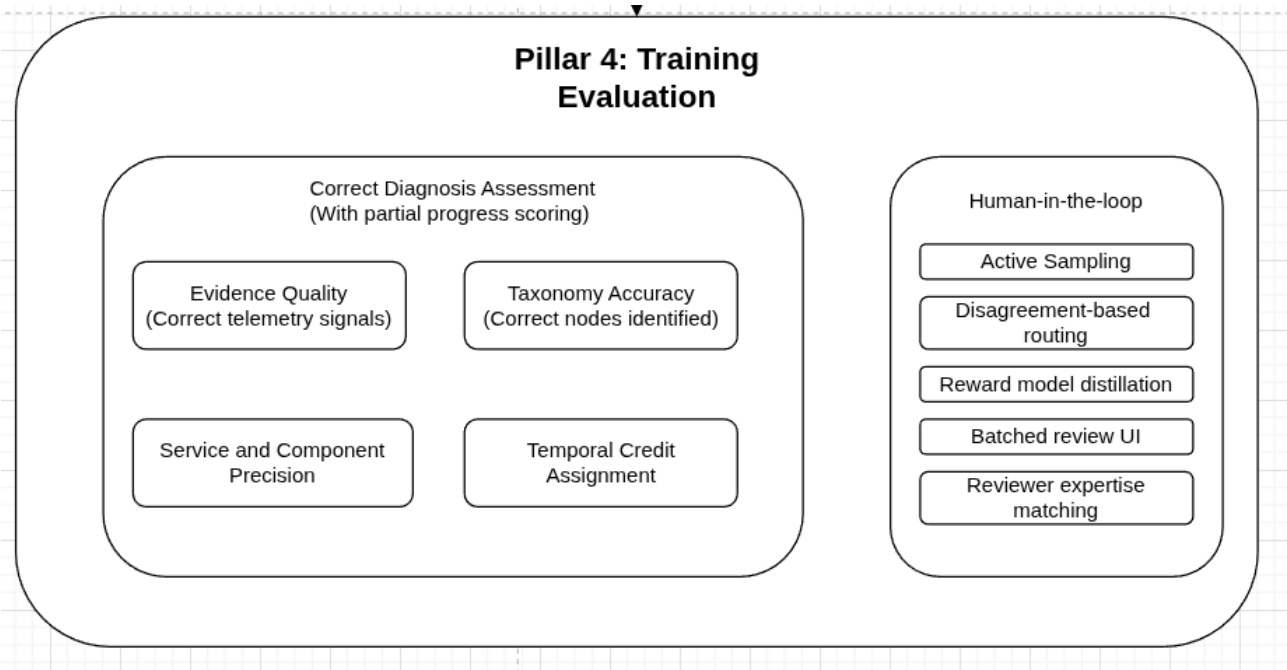
Layer 3: Full Cluster Simulation (1-10% of training episodes)

What it is: Ephemeral Kubernetes clusters (k3d or vcluster) running representative real applications with production-like configurations. This layer is primarily used for: validating that L1 and L2 scenarios faithfully represent real failure signatures, generating ground-truth telemetry datasets for rare or complex failure modes, and final evaluation of agent capabilities before deployment.

Episode time: ~5 minutes. Not suitable for online RL training loops.

Cost: ~$5–10 per episode. Budget for 50–100 L3 episodes/week for validation and calibration.

Pillar 4: Evaluation & Reward Design (see “Training Evaluation”



SRE diagnosis is not a classification problem with a single correct answer. Root cause analysis is inherently a Bayesian inference problem: given observed signals, what is the most probable explanation? Multiple explanations may be consistent with the evidence. Remediation effects are delayed and noisy. A simple "did you get it right?" reward will present inaccurate “correct” states.

How do we get to the “correct” diagnosis?

Diagnosis correctness is scored on three dimensions:

1. **Taxonomy accuracy**: Did the agent identify the correct taxonomy nodes? Scored by F1 against the ground truth set, weighted by node confidence. This handles multiple valid root causes naturally — the agent is rewarded for identifying all of them.
2. **Evidence quality**: Did the agent cite the right telemetry signals? Scored by overlap between agent-cited signals and injected signals. An agent that correctly identifies a memory leak but cites CPU metrics as the evidence gets partial credit.
3. **Localisation precision**: Did the agent identify the right service? The right component within the service? Scored hierarchically — service-level correct + component-level wrong earns more than pure miss.

Temporal Credit Assignment

Real-Time Action Tracking

Every action the agent takes is logged with a timestamp and system snapshot. This creates a complete record of the system state before and after each action, which feeds a counterfactual simulator.

Shadow Simulation Running in Parallel

Alongside the live system, a lightweight shadow environment mirrors the system state continuously. When the agent takes an action, the shadow environment branches into two paths:

- Path A – continues with the action applied
- Path B – continues as if the action never happened

Scoring Pipeline

As the system recovers, a scoring service:

1. Compares recovery time on both paths
2. Assigns a credit score to each action
3. Feeds scores back into the AI's training loop

Guardrails in Production

- Shadow simulations are capped in compute cost to avoid slowing down the live system
- At higher complexity levels, a pre-trained estimator replaces full simulations to keep costs manageable
- All counterfactual scores are auditable, so engineers can review why the AI valued certain actions over others

Human-in-the-Loop Evaluations

Human evaluation is expensive. We can minimise it through:

1. **Active sampling**: Routes to human review only scenarios where automated reward confidence is below 0.6. Confidence is estimated by the variance across an ensemble of reward models.
2. **Disagreement-based routing**: When the LLM-based reward model and the rule-based reward model disagree by >0.3, route to human.
3. **Reward model distillation**: Human ratings are used to fine-tune the automated reward model. Over time, fewer disagreements → fewer human reviews needed. Target: <5% of episodes require human review at steady state.

4. **Batched review UI**: Human reviewers see a structured interface showing the scenario, agent trajectory, and auto-generated reward rationale. They approve/adjust, not evaluate from scratch. Reduces review time to ~2 minutes per case.
5. **Reviewer expertise matching**: Route security-related scenarios to security engineers, database scenarios to DBA reviewers. Reduces back-and-forth corrections.

Pillar 5: Test Case Generation & Coverage

From Indexed Incident Data to Reproducible Test Cases

The conversion pipeline has four stages:

Stage 1 — Incident Decomposition: An LLM extracts structured fields from the `IncidentRecord`: service topology (what services were involved), fault injection point (where the failure started), symptom cascade (how it propagated), observable signals (what metrics/logs/traces were visible), and resolution steps (what fixed it).

Stage 2 — Template Matching: The extracted structure is matched to a scenario template — a parameterised description of a failure mode. Templates are hand-crafted for each taxonomy leaf node (45 templates at launch). A template specifies what parameters are needed and which parameters can be varied to generate test case variants.

Stage 3 — Instantiation: The incident data fills in the template parameters. Multiple incidents mapping to the same template produce multiple instantiations. Parameters not present in the incident (e.g., specific latency values if only "elevated latency" is mentioned) are filled with distributions learned from historical data.

Stage 4 — Validation: The instantiated scenario is run through L1 simulation and the resulting telemetry is checked against sanity rules (e.g., a memory leak scenario should show monotonically increasing RSS; a network partition should show 100% packet loss on the affected link). Scenarios that fail sanity checks are flagged for manual review.

Each test case is stored with:

- A deterministic ID (hash of template + parameters)
- The source incident(s) it was derived from
- The taxonomy node(s) it exercises
- Difficulty rating (derived from average agent success rate after 1000 training episodes)
- Generation origin (template-based vs. LLM-synthesised vs. mutation)

Corpus Strategy for 80% Coverage

Phase 1: High-frequency, high-confidence nodes. Focus on a selection of taxonomy nodes that account for ~50% of incidents. Generate a large number of test cases per node from real incidents.

Phase 2: Combined scenarios. Most real-world incidents involve multiple failure modes interacting. Generate multi-node scenarios by composing two or three taxonomy nodes simultaneously. This multiplicatively expands coverage and tests the agent's ability to reason about concurrent failures.

Phase 3: Long-tail and novel scenarios. Use LLM synthesis to generate scenarios for under-represented nodes. Validate against real incidents as they appear. Target 10+ test cases per leaf node.

Generating Novel Scenarios for Generalisation Testing

Mechanism 1 — Parameter Mutation: Take an existing test case and systematically vary parameters outside the training distribution.

Mechanism 2 — Topology Switching: Take a failure mode from one service architecture and instantiate it in a completely different topology.

Mechanism 3 — LLM-Guided Adversarial Generation: Given the agent's current policy, prompt an LLM to generate scenarios specifically designed to exploit known weaknesses.

Measuring Coverage and Prioritising Gap-Filling

To systematically track and improve agent performance across failure categories, a coverage matrix is maintained that maps each taxonomy leaf node against the number of real-world incidents observed, the number of test cases available, and the agent's win rate on those cases.

Gap priority is then calculated as a function of these variables. Leaf nodes with low incident counts, sparse test coverage, and poor agent win rates are scored more critically, particularly when the underlying failure type is known to occur frequently in production.

Taxonomy Node	# Incidents	# Test Cases	Agent Win %	Gap Priority
net.partition	234	89	72%	LOW
app.cascade_failure	891	312	68%	LOW
net.bgp_flap	12	3	31%	HIGH
compute.clock_skew	8	2	44%	HIGH
app.livelock	4	1	22%	CRITICAL

How the RL Loop Closes

The RL loop improves the agent over time against unseen scenarios through three mechanisms:

Curriculum progression: Scenarios are ordered by difficulty (estimated from agent win rate). New agents start on easy, well-represented scenarios. As win rate on a node exceeds 80%, that node's

weight in training batches decreases and harder/rarer scenarios are introduced. This prevents the agent from over-optimising for easy cases.

Adversarial scenario generation: The scenario creation service continuously produce new scenario variants the agent hasn't seen. These are added to the held-out evaluation set first. Once 10+ agent versions have been evaluated on them, they're promoted to the training set. This ensures the evaluation set is always ahead of the training set.

Taxonomy-driven exploration: When the coverage tracker identifies a gap node (high priority, low agent win rate), it triggers a targeted generation burst. The training loop temporarily upweights episodes from that node. This ensures the agent doesn't plateau by over-training on well-covered nodes while ignoring rare but important failure modes.

Build Order

MVP (First 8 Weeks)

The MVP has a single goal: close the loop end-to-end, even if each component is rough. A closed loop that trains an agent and shows measurable improvement is worth more than any individual component at production quality.

- Ingest pipeline from 1 source (GitHub post-mortems repo). Normalisation schema.
- Taxonomy covering the top failure modes by frequency. LLM-based classifier with no fine-tuning.
- L1 telemetry generator for a select number of taxonomy nodes. Scenario compiler that takes an incident + template and produces a reproducible episode.
- Reward assignment service with taxonomy accuracy + localisation score. First RL training run against open-sre-agent. Measure baseline win rate per taxonomy node. Target: closed loop runs, agent shows improvement over 1000 training episodes.

Deferred (Post-MVP)

Component	Why Deferred
L2/L3 simulation layers	L1 is sufficient to close the RL loop; L2/L3 add fidelity but not fundamental capability
Adversarial scenario generation	Needs a trained agent to red-team against but not needed to prove capability
Full node taxonomy	Start with highest-frequency nodes; expand as coverage proves value
Temporal credit assignment with counterfactual simulator	Computationally expensive; simpler return-to-normal reward is sufficient for MVP

Component	Why Deferred
Human review queue UI	Manual spreadsheet-based review is acceptable at MVP scale (manageable number of classifications/week)
Fine-tuned reward model	Fine-tune once 500+ human-reviewed episodes exist

Integration with Open SRE Agent

Because this training environment is an environment that Open SRE Agent operates within, it needs to have an API that the agent can call that supplies Open SRE Agent with realistic data. These components are then accessed in the test suites.

Open SRE needs to have access to the following:

1. Episode manager: able to start the scenario that returns the initial observation or alert, register the steps the agent takes in the environment for tool calls and remediation attempts, and then render the human-readable replay of the episode that is used for debugging, human review, and reward training of the model.
2. Tool shims: simulate the tools and the feedback those tools provide. Each tool that the agent has access to will have a shim that returns a response formatted identically to what the real tool would return – all generated through the test case generator.